
Axini Laboratory

Axini

October 8, 2025

Contents

1	The Coffee Machine	2
1.1	Exploring AMP	2
1.2	Extending the model	3
1.3	Testing the SUT	3
1.4	When life gives you lemons	3
2	SmartDoor	5
2.1	Modeling	5
2.2	Skeleton	5
2.3	Open and close: good weather	6
2.4	Open and close: bad weather	6
2.5	Locking and unlocking	6
2.6	Testing	7
3	SmartDoor Interface Requirements Specification	8
3.1	Requirements	8
4	Report	10
5	Grading	11

Introduction

In this laboratory, you will carry out some practical exercises with a state-of-the-art Model Based Testing (MBT) tool: the Axini Modeling Platform (AMP). AMP is an industrial-strength MBT tool, developed by Axini¹ during the last decade.

The modeling language used in AMP is the Axini Modeling Language (AML). The semantics of AML models is defined upon Symbolic Transitions Systems (STS), a data-extension of Labeled Transition Systems (LTS). For this laboratory we will mostly use the LTS part of AML.

AMP is an integrated development Platform for AML models. Models can be edited, visualized, simulated, and most importantly, tested against a system under test (SUT). All test results are automatically saved. AMP uses an *online testing* approach, which means that test cases are generated on-the-fly while testing the SUT, rather than beforehand.

AMP is a web application. For this laboratory, an online version of AMP is available at <https://course02.axini.com>. Only the browsers *Google Chrome* and *Firefox* are fully supported, but other modern browsers should work too.

To become familiar with AMP, you will first do an introductory exercise with a Coffee Machine model and SUT. This will give you some hands-on experience with modeling and testing. Afterwards, you will be ready to do a small project with AMP, where you will model a remote controlled SmartDoor from scratch using a specification document and test various implementations of that system.

¹ <https://www.axini.com/>

1. The Coffee Machine

In this exercise you will play around with a simple model of a Coffee Machine which dispenses coffee, tea and lemonade. You will see the core features of AMP, extend a small AML model and test an implementation of the Coffee Machine using the model you created.

1.1 Exploring AMP

When you log in for the first time, you will arrive at the test page with the message ‘This test set has no test runs yet’.

Your environment should contain multiple projects to work in. Select the Coffee Machine project using the navigation bar at the top of the screen. In this navigation bar, you can also select a test set, which are used to manage different testing configurations. The Coffee Machine project contains only a single test set ‘Coffee Machine’.

We will start by exploring the main features of AMP.

1.1.1 Model

Press the ‘Model’ button in the sidebar to enter the model editor. This is the integrated modeling environment of AMP where you can manage your AML models. To get you started, your model repository already contains a beginning of the model of the Coffee Machine in the model part `coffee_machine.aml`.

Inspect this model carefully. Do you understand everything?

1.1.2 Visualize

Press the ‘Visualize’ button. This will open the visualization page in a new window in your browser. In this new window, the STS of the model is shown. Can you relate the visualized STS to the textual model?

1.1.3 Explore

Select the main tab of AMP again and press the ‘Explore’ button. This will open the explorer page, a simulator for the model. AMP will offer a menu of actions it can perform. In the initial state, two actions are possible: we can select the `button_coffee` stimulus or let time pass using the `tick` action. Under ‘State vectors’ the current state of the process `coffee machine` is shown: `coffee machine_start`, the initial state. Advance on the `button_coffee` stimulus to simulate pressing this button. AMP will advance to the next state and will offer a new set of actions to choose from. At the bottom of the page, the ‘Explored Trace’ shows the actions executed.

Note

The explorer only uses the model: the actual Coffee Machine implementation is not used yet.

1.1.4 Test

Now we are ready to test an implementation of the Coffee Machine. Press the ‘Test runs’ button. This brings us back to the test page. Press the ‘Start’ button to test whether the SUT conforms to the model. You will see a chart which shows the transition coverage of the model. Below the chart, a summary of the test cases that have been executed is shown. This project is configured in such a

way that a test run consists of three test cases, each of ten steps each. All three test cases should have passed.

1.1.5 Test case details

On the test page, select one of the test cases. A detailed trace of the steps is shown. Note that the trace is presented in the same way as an exploration of the explorer. This test case is derived from the model. Can you relate the two?

1.1.6 Coverage

After the test, AMP can give information on the state coverage and transition coverage of the test run. Press the 'Coverage' button on the test run page to go to the coverage page.

The coverage page shows that the test run has covered all states and transitions of the model. Press the Visualize button, which opens a page with the STS of the model where the states and transitions are either covered (green) or not covered (red). Due to the 100% coverage, all states and transitions are green. Remember that behavior of SUT that is not included in the model is not tested and cannot be reported on.

1.1.7 Help

Finally, if you want to learn more about AMP or AML, the tool provides ample documentation, accessible from the top right corner.

1.2 Extending the model

The model currently only describes how to get coffee. The actual Coffee Machine also dispenses tea when `button_tea` is pressed. As you can see in the generated test cases, this behavior is therefore not tested either².

- Extend the model with the behavior to dispense tea. When done, check the visualization of your model. Does it look as expected?

Hint

You can use the choice keyword from AML to model alternative paths.

1.3 Testing the SUT

- Test the SUT again with your extended model. The test run should pass. If not, inspect your model carefully and fix any mistakes.
- Inspect the generated test cases and the coverage page to confirm that the added behavior is tested.

1.4 When life gives you lemons

The Coffee Machine can also dispense *lemonade* when the corresponding button is pressed. Let us add this behavior to the model as well.

² We allow our models to be *underspecified*, which means not all possible behavior has to be included. This is a very useful property when testing, but also requires care to ensure we test everything we want to.

- We have not yet defined labels in the model for stimulus `button_lemonade` and response `lemonade`. Add them now. Note that the label names cannot be chosen freely; these label names are implemented in the adapter we are using to test the actual Coffee Machine SUT. Therefore, the model must use the same label names³.
- When you are done, test the SUT again. Inspect the executed test cases carefully. What does the model expect after `button_lemonade` is pressed? How does the SUT actually react?
- Inspect a number of different test cases. Can you discern a pattern in the behavior of the SUT?

³ When you make your model before developing the adapter it will be the other way around. In practice, both are built around the same time when testing a new system.

2. SmartDoor

Now that you are more familiar with AMP, we can continue to the main event of this laboratory. You will do a small MBT project in which you will model and test various implementations of a remote controlled door called SmartDoor. This project should give you a good feel for how MBT works in practice.

In AMP, you can select the SmartDoor project using the navigation bar at the top. This project contains multiple Test Sets, each representing a different implementation of the SmartDoor. For now, choose the Axini test set.

2.1 Modeling

In this exercise, you will write a model in AML from scratch.

Informally, the door has the following features: it can be opened, closed and locked. When the door is locked, a passcode is set. A separate [SmartDoor Interface Requirements Specification](#) (chapter 3, page 8) (IRS) document is provided that contains more details.

In the following sections, we will guide you through modeling the behavior of SmartDoor. After each step, we recommended checking the visualization to see if your model ended up the way you intended. Moreover, the SmartDoor implementation by Axini has already been tested with MBT. You can consider this implementation to be correct and use it to validate your model.

2.1.1 The adapter

AMP makes use of adapters as a bridge between the model and the SUT. An adapter can translate messages back and forth between the symbolic representation of the model and the technical formats used by the different implementations. The adapter needed to test the various SmartDoor implementations has been provided for you.

Since the adapter translates symbolic labels from the model, it requires built-in knowledge of the names of channels, labels and label parameters used in the model. Therefore, follow any naming instructions from the IRS carefully. Moreover, the adapter also defines which interactions with the SUT are possible. As you will see, some cases cannot be tested with the given adapter.

The door will always begin *closed* and *unlocked*. The adapter ensures the SUT is in this state when a test case is started.

2.2 Skeleton

First, we will write a skeleton for the SmartDoor model, making use of what you have learned during the Coffee Machine exercise.

- Define a process for the door and an external channel named `door`.
- Define the labels needed for the model. The label and channel names cannot be chosen freely as they must be known by the adapter used by the SUT. The configured adapter implements the labels as written in the specification on the channel named `door`. Start with the stimuli `open` and `close` and the responses `opened` and `closed`.
- Optionally, you could also define the other stimuli and responses at this stage. This might save some time later on.
- Set a global timeout for responses.

2.3 Open and close: good weather

When modeling a new system, we generally start small and extend the model over time. The SmartDoor model is well suited for this approach.

- Let us start with modeling some of the good weather behavior. This is the intended usage of the system. Add an AML fragment that describes the opening and closing behavior of the door. You can use the same modeling concepts used in the Coffee Machine model.

2.4 Open and close: bad weather

Now we will model how the SUT should react to bad weather behavior of the environment. Bad weather behavior is concerned with behavior which should not be offered by well behaved clients of the SUT. These are typically stimuli which are not allowed in certain states or invalid data values.

- Extend the model so that it can receive an `open` stimulus when the SmartDoor is already opened and it can receive a `close` stimulus when the SmartDoor is already closed. What are the expected responses in these cases?

Note

You will need to define a new response in your model. Otherwise, you will see the following error: No label is defined that matches with X.

Hint

You can use the `repeat` keyword instead of the `choice` keyword to model staying in the same state without the need of a `goto`.

2.5 Locking and unlocking

Now we are going to add the locking and unlocking behavior of the specification to the model. Here, you will see the data aspect of Symbolic Transition Systems come into play.

- Model the new labels, states and transitions for locking and unlocking the door. You can ignore the passcode at this stage. Do not forget to model the relevant bad weather behavior.
- Define a *label parameter* named `passcode` of type `:integer` for the `lock` and `unlock` stimuli. The adapter recognizes this label parameter name and uses it when locking and unlocking the door⁴.
- Use the `constraint:` option of the `receive` keyword to constrain the value of the passcode on the relevant transitions. Read the specification carefully to ensure that all possible behavior is covered. For example, what should happen when the door is locked with an invalid passcode?

Hint

You can use a state variable of type `:integer` to store the selected passcode in your model. The value of this variable can be referenced later to accept or reject the passcode of the `unlock` command.

⁴ The adapter will translate the integer representation of the passcode (e.g. 42) into the four digit format (e.g. 0042). This makes it easy to model the domain of passcodes. However, the case where the given passcode is too short cannot be tested this way.

2.6 Testing

When your model is completed, you can start testing the various SmartDoor implementations. Within the SmartDoor project, you will find a test set for each manufacturer:

1. Axini
2. Besto
3. Logica
4. OnTarget
5. quickerr
6. SmartSoft
7. TrustedTechnologies
8. univerSolutions
9. XtraSafe

As stated before, the implementation of Axini is intended to be correct, so you can use it to validate your model. All test cases should pass when testing this SUT.

When a test case fails, AMP has detected a difference between the observed and the modeled behavior. It is up to you to analyze both and conclude whether the model or the SUT is in the wrong. When in doubt, the specification can often provide the answer. If you conclude that the SUT is in the wrong, you have found a bug in the implementation. Note that implementations might contain multiple bugs, so do your analysis carefully.

When we find bugs in the real world, we report them in such a way that a product owner has sufficient information to evaluate our findings, and developers have enough information to fix them. This often involves snippets from the log file to catch the SUT red handed.

For you, it is enough if you can answer the following questions.

1. In what state(s)/situation(s) does the bug occur?
2. What is the expected behavior?
3. What behavior is observed instead?
4. What is your hypothesis of how the bug works?

Note

Since you are testing multiple implementations, it is recommended to track the found bugs carefully. You can add tags to categorize (failed) test cases for easier analysis.

Note

The Axini Modeling Language guide accessible via the help page of AMP can provide additional tips on dealing with bugs and making alternative behavior toggleable in your model using Ruby macros.

3. SmartDoor Interface Requirements Specification

SmartDoor is a family of remote controlled doors controlled through a mobile app. The various SmartDoor products are produced by different manufacturers. By implementing the same SmartDoor protocol, users can control all these products through the same app.

This document describes the requirements that all SmartDoor implementations must adhere to. This ensures interoperability and safety standards are maintained. In this document “the door” refers to the SmartDoor implementation and the “controller” refers to the (mobile) application controlling it.

3.1 Requirements

3.1.1 Dialog requirements

The following section describes the requirements specifying the communication between the controller and the door.

DIALOG-01

The communication between the controller and the door follows a command-response pattern. Each dialog between the controller and the door consists of a single command followed by a single response.

DIALOG-02

The controller may initiate a dialog. It may only send command messages.

DIALOG-03

The door may not initiate a dialog. It may only send response messages.

DIALOG-04

The door must always respond to a command from the controller with a response message.

DIALOG-05

After receiving a command, the door must respond within 0.5 seconds.

DIALOG-06

The following messages of the SmartDoor protocol are command messages:

- open request the door to open
- close request the door to close
- lock request the door to lock
- unlock request the door to unlock

DIALOG-07

The following messages of the SmartDoor protocol are response messages.

- opened confirm that the door has been opened
- closed confirm that the door has been closed
- locked confirm that the door has been locked
- unlocked confirm that the door has been unlocked
- invalid_command signal that an invalid command has been received
- invalid_passcode signal that the provided passcode was invalid
- incorrect_passcode signal that the provided passcode was incorrect

DIALOG-08

When a command is accepted, the door must respond with the corresponding confirmation message.

DIALOG-09

When a command is refused, the door must respond with the `invalid_command` message.

DIALOG-10

When an unknown command is received, the door must respond with the `invalid_command` message.

3.1.2 Behavior requirements

The following section describes the requirements specifying how the door should act on commands. The door must accept or refuse commands based on its current state.

BEHAVR-01

When the door is closed and unlocked, it may be opened.

BEHAVR-02

When the door is opened, it may be closed.

BEHAVR-03

When the door is closed and unlocked, it may be locked.

BEHAVR-04

When the door is closed and locked, it may be unlocked.

BEHAVR-05

All other commands must be refused.

3.1.3 SecureLock requirements

The following section describes the requirements specifying the SecureLock security feature present in all recent SmartDoor products. This feature prevents unauthorized access by setting a four digit passcode when the door is locked. The door can only be unlocked with that same passcode.

SECLOC-01

The lock and unlock commands are accompanied by a four digit passcode.

SECLOC-02

The four digit passcode must be in the range 0000 to 9999. Other passcodes are considered invalid.

SECLOC-03

When the lock command is used with a valid passcode, the door can only be unlocked with an unlock command containing that exact passcode. Other passcodes are considered incorrect.

SECLOC-04

When the lock command is used with an invalid passcode, the command must be refused and then door must return the `invalid_passcode` response.

SECLOC-05

When the unlock command is used with an incorrect passcode, the command must be refused and then door must return the `incorrect_passcode` response.

SECLOC-06

When the unlock command is used with an invalid passcode, the command must be refused and the door must return the `invalid_passcode` response.

SECLOC-07

When the unlock command is used three times with an incorrect passcode, the door must shut off and not respond to any commands until it is restarted physically.

4. Report

Write a short report in which you reflect on your experience with MBT. The following questions can get you started.

- How does the AML model compare to the textual specification? What are the strengths and weaknesses of each?
- Summarize the promised benefits of mbt. Based on your experiences in this laboratory, do they hold up in practice?
- What disadvantages and limitations do you see in MBT? Do you see threats to correctness in mbt with AMP?
- How does MBT compare to other testing approaches? In what situations would you apply MBT?

You can include or append your test results in the report. Don't forget to submit your AML model (DSL text) of the SmartDoor as well.

5. Grading

Grading will be done as follows:

- **(4 points)** *Modeling*: we will grade your AML model on whether it includes all behavior of the SmartDoor.
- **(3 points)** *Testing and Analysis*: find, describe and perform analysis on all *easy bugs* and analyze what might have caused them.
- **(3 points)** *Testing and Analysis*: find, describe and perform analysis on all *hard bugs* and analyze what might have caused them.

A **bonus point** is awarded for teams that find all bugs in the SmartDoor implementations.

The **page limit** for describing all the bugs is 1 A4. Exceeding this page limit will result in a deduction of 1 point per additional page.